



INDIA.RUNS

Build what next India runs on

Team Name : Tina Aswani

Team Leader Name : Tina Aswani

Problem Statement : Intelligent Candidate Discovery & Ranking for AI/ML Hiring

Solution Overview

- What is your proposed solution?

An end-to-end CPU-only pipeline that processes 100,000 candidate JSON profiles and ranks the Top 100 candidates for a Senior AI Engineer role – using explainable rule-based scoring, no LLM calls, no GPU, no external APIs.

- What differentiates your approach from traditional candidate matching systems?

Unlike keyword matchers or LLM-based systems: transparent weighted scoring, honeypot/fake profile detection, behavioral signal analysis, and grounded reasoning – all running in ~81.5 seconds on CPU alone.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

JD signals: Senior AI Engineer – Python, PyTorch/TF, LLM experience, MLOps, system design, research depth. Both hard skills and seniority indicators are weighted.

- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Key signals: skill overlap score, years of relevant experience, role seniority, career trajectory, education tier, publication/OSS activity, job stability, and red-flag detection (impossible timelines, skill stuffing, honeypot patterns).

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

All 100,000 profiles are parsed and feature-extracted in a single pass (features.py). Each candidate receives a composite weighted score via scoring.py. Top 100 are selected and written to ranked CSV.

- What models, algorithms, or heuristics are used?

Transparent rule-based heuristics using pandas. No ML model — fully interpretable. Skill match via token overlap, experience scored by recency-weighted tenure, seniority mapped from role titles.

- How are multiple candidate signals combined into a final ranking?

Weighted sum: Skill Match (35%) + Experience Quality (25%) + Seniority (15%) + Education (10%) + Behavioral Signals (10%) + Stability (5%). Red-flag penalties applied before final sort.

Explainability & Data Validation

- How are ranking decisions explained?

Each candidate's score is broken down per dimension. A grounded reasoning string is generated from actual profile data – no hallucinated claims, every reason traceable to a field in the JSON.

- How do you prevent hallucinations or unsupported justifications?

No LLM is used during ranking – zero hallucination risk. All justifications are template-filled from structured feature values. `self_check.py` validates output integrity before submission.

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

Honeypot detection flags: impossible date overlaps, skill count anomalies (> 80 skills), implausible career timelines, and duplicate/templated profiles. Flagged candidates receive score penalty or disqualification.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?
1. Load 100K JSON profiles → 2. Parse & clean messy fields (features.py) → 3. Extract structured features (skills, tenure, seniority, education, behavioral signals) → 4. Honeypot detection pass → 5. Weighted scoring (scoring.py) → 6. Sort & select Top 100 → 7. Generate reasoning strings → 8. Export ranked CSV → 9. Validate with self_check.py

System Architecture

- **Entry Point:** rank.py – orchestrates full pipeline, CLI entrypoint
- **Feature Layer:** features.py – parses JSON, extracts skills, tenure, seniority, education, behavioral signals
- **Scoring Layer:** scoring.py – weighted composite score, red-flag penalties, Top-100 selection
- **Validation Layer:** self_check.py – validates output schema, rank order, required fields
- **Demo:** redrob_sandbox.ipynb – Colab notebook for interactive exploration and output visualization

Results & Performance

- What results or insights demonstrate ranking quality?

Top-100 candidates show: 8+ years average AI/ML experience, 90%+ required skill coverage, verified senior-level roles (Lead/Principal/Staff), and clean honeypot-free profiles. Output is fully explainable with per-candidate reasoning strings.

- How does your solution meet the challenge's runtime and compute constraints?

Runtime: ~81.5 seconds for 100,000 profiles on CPU only. No GPU. No network calls. Python 3.11 + pandas only. Fully offline execution – meets all challenge constraints with margin.

Technologies Used

- Stack selected for constraint-compliance, transparency, and performance:
- **Python 3.11** – core language; chosen for speed, ecosystem, and challenge compatibility
- **pandas** – vectorized feature operations, fast sorting and filtering at 100K scale
- **Google Colab** – sandbox demo environment; accessible, reproducible, no setup required
- **GitHub** – version control and public submission at github.com/tinaaswanii/-redrob-hackathon
- **No external APIs / No GPU / No LLM** – fully self-contained, offline-safe, constraint-compliant

Submission Assets

All project assets available at:

GitHub Repository: <https://github.com/tinaaswanii/-redrob-hackathon>

Colab Notebook: colab.research.google.com/github/tinaaswanii/-redrob-hackathon/blob/main/redrob/redrob_sandbox.ipynb

Key Files: rank.py | features.py | scoring.py | self_check.py | redrob_sandbox.ipynb

Output: ranked_candidates.csv – Top 100 candidates with scores, rank, and reasoning strings



INDIA.RUNS

Build what next India runs on

THANK YOU